# TriSense: A Single Wearable Edge AI Device with Three Switchable Accessibility Modes

Neelabja Mukhopadhyay
*School of Computer Science and Engineering*
*VIT Bhopal University*
Bhopal, Madhya Pradesh, India
Reg. No. 23BCE11053

Yash Aman
*School of Computer Science and Engineering*
*VIT Bhopal University*
Bhopal, Madhya Pradesh, India
Reg. No. 23BCE11558

Aryan Dev Jha
*School of Computer Science and Engineering*
*VIT Bhopal University*
Bhopal, Madhya Pradesh, India
Reg. No. 23BCE11807

Mohammed Farhan Khan
*School of Computer Science and Engineering*
*VIT Bhopal University*
Bhopal, Madhya Pradesh, India
Reg. No. 23BCE11656

Geet Basantani
*School of Computer Science and Engineering*
*VIT Bhopal University*
Bhopal, Madhya Pradesh, India
Reg. No. 23BCE11118

***Abstract*—People who are blind, deaf, or mute are usually served by separate assistive products, each built for a single need. A person who cannot see is expected to buy one device for obstacle avoidance. A person who cannot hear needs a different captioning tool. A person who cannot speak needs yet another sign language translator. This separation raises the total cost of ownership, adds extra weight and learning effort for the user, and becomes impractical for anyone who happens to face more than one of these challenges at once. Many of the existing captioning and translation tools also depend on cloud based speech and language services, which stop working the moment network connectivity is lost, a condition that is common across large parts of India and other developing regions. This paper presents TriSense, a single wearable device built around a Raspberry Pi 4 that runs entirely at the edge and offers three independent, software switchable accessibility modes from one shared piece of hardware. Blind Mode performs obstacle detection using an ultrasonic sensor array and reports danger through directional haptic pulses and short spoken alerts. Deaf Mode performs fully offline speech to text captioning and renders the result on a small OLED screen. Mute Mode observes hand gestures through a camera and speaks the recognized sign aloud using a lightweight neural classifier. A coordination layer named ModeManager, supported by a resource locking component called ResourceGuard, allows the three modes to share six hardware resources, namely the speaker, the camera, the microphone, the display, and two groups of GPIO pins, without ever leaving the system in a half switched or conflicting state. Blind Mode and Deaf Mode, treated as the two mandatory deliverables of this project, are software complete and pass their respective synthetic and modular test suites. Mute Mode, treated as a protected stretch goal that was never allowed to take time away from the two mandatory modes, is fully wired into the coordination layer and has been trained on real data from the WLASL dataset, currently reaching 33.33 percent validation accuracy across a one hundred word vocabulary. A documentation audit carried out before any hardware was connected also uncovered and resolved a three way conflict between GPIO pins assigned to the ultrasonic sensors and the pins used by the shared audio bus. Taken together, these results show that a single, low cost, fully offline wearable can plausibly serve three different accessibility needs at once, provided that shared hardware access is coordinated carefully at the software level.**

***Index Terms*—assistive technology, wearable computing, edge artificial intelligence, obstacle detection, offline speech recognition, sign language recognition, resource coordination, Raspberry Pi**

## I. Introduction

### A. Background and Motivation

Assistive technology has historically grown in narrow, single purpose channels. A cane fitted with an ultrasonic rangefinder helps a blind person avoid obstacles but does nothing for someone who cannot hear a conversation. A captioning application running on a phone helps a deaf person follow speech but is of no use to someone who cannot see the screen. A sign language recognition tool helps a mute person communicate but is rarely combined with either of the other two capabilities. Because these products are designed and sold independently, a household or an institution that needs to support more than one kind of disability ends up purchasing, charging, carrying, and learning several devices at once. The financial and cognitive cost of this fragmentation falls hardest on exactly the population that assistive technology is meant to help.

A second and equally important problem is connectivity. Several of the more capable captioning and translation systems available today send audio or video to a cloud service for processing. This design choice improves accuracy in favorable

conditions but makes the device useless the moment network coverage becomes weak or unavailable. Large parts of India, along with many other developing regions, still experience patchy or absent mobile data coverage in rural areas, inside buildings with poor signal penetration, and during outages. An assistive device that stops functioning under exactly these conditions offers a poor guarantee of reliability to a user who may depend on it for basic safety.

These two observations, the cost of fragmentation and the fragility of cloud dependence, motivated the central question behind this project. Could a single wearable device, built from commodity edge hardware and running every model entirely on device, responsibly cover three different accessibility needs without weakening the reliability of any one of them. TriSense is the answer developed and evaluated in this paper.

### B. Problem Statement

Building a single device that serves three different accessibility needs is not simply a matter of installing three separate applications on one computer. The modes compete for the same physical resources. The speaker, for instance, must be available to both Blind Mode for spoken alerts and Mute Mode for spoken sign output, yet only one of these subsystems can safely hold it at any given moment. The camera is needed only by Mute Mode but must be released cleanly when the user switches away from that mode. The GPIO pins used to read the ultrasonic sensors and drive the haptic motors must not collide, electrically or logically, with the GPIO or bus lines used by the microphone, the display, or the speaker amplifier. Any system that ignores this coordination problem risks silent failures that are extremely difficult to diagnose once physical hardware is involved. The core engineering problem addressed in this paper is therefore twofold. First, each of the three accessibility pipelines must be implemented correctly and verified on its own terms. Second, a coordination layer must be designed that lets these three independently built pipelines share one set of hardware resources safely, predictably, and without requiring any of the three to be rewritten around the others.

### C. Objectives

The project set out to design, implement, and verify a single wearable edge AI device, TriSense, built on a Raspberry Pi 4 platform, that offers three independently switchable accessibility modes.

Blind Mode was defined as real time obstacle detection using an array of ultrasonic sensors, reporting danger through directional haptic feedback and short spoken alerts. This mode was treated as a hard, non negotiable deliverable of the project.

Deaf Mode was defined as fully offline, real time speech to text captioning, rendered live on a compact OLED display. This mode was also treated as a hard, non negotiable deliverable.

Mute Mode was defined as sign language to speech translation, using camera based hand landmark tracking paired with a lightweight neural classifier. Unlike the first two modes, Mute Mode was explicitly scoped as a protected stretch goal, meaning that it was allowed to grow beyond its original ambition only as long as doing so never consumed schedule time or engineering resources that had been committed to Blind Mode or Deaf Mode.

A further objective sat above these three modes. The team needed a coordination layer capable of switching safely between the modes while they shared five categories of hardware, the speaker, the camera, the microphone, the display, and the GPIO lines used for sensing and haptics, without ever creating a resource conflict. Every software module produced for the project was also required to pass modular and synthetic end to end testing before any physical hardware was connected, so that defects could be caught and fixed while they were still cheap to fix.

### D. Contributions

This paper makes the following contributions. It presents the design and implementation of three independently verified accessibility pipelines running on a single embedded platform. It describes a resource coordination layer, built around a five step stop, release, verify, acquire, and start protocol, that allows these three pipelines to share hardware safely through thin adapter classes rather than through invasive changes to each pipeline's internal code. It reports concrete, reproducible verification results for all three modes, including a deterministic synthetic test for Blind Mode, a modular test suite for Deaf Mode, and an end to end integration test for Mute Mode. Finally, it documents a real engineering defect, a three way GPIO pin conflict, that was found through a systematic documentation audit before hardware bring up, illustrating the practical value of cross checking pin assignments against a shared bill of materials rather than relying on unit tests alone.

### E. Paper Organization

The remainder of this paper is organized as follows. Section II reviews prior work in ultrasonic obstacle detection, offline speech captioning, and MediaPipe based sign language recognition, and positions TriSense relative to that work. Section III describes the front end and back end requirements of the system along with the hardware and software bill of materials. Section IV presents the overall system architecture, including the ModeManager and ResourceGuard components. Section V explains the working principle of each of the three modes in detail. Section VI reports the results obtained from modular and synthetic testing, along with a discussion of the GPIO conflict that was discovered and resolved during development. Section VII discusses the current limitations of the project and the work planned for the next phase. Section VIII concludes the paper.

## II. Related Work

Prior research in assistive technology for blind, deaf, and mute users is extensive, but almost every system reviewed here was designed to solve exactly one of these three problems. This section reviews representative work in each domain and explains how TriSense positions itself against that body of work.

### A. Obstacle Detection for the Visually Impaired

Ultrasonic sensing has long served as a low cost and dependable basis for obstacle detection aimed at blind and visually impaired users. Sweekat et al. proposed a waist belt mounted electronic travel aid that used a network of ultrasonic sensors to detect obstacles up to three hundred centimeters away across three directions, with particular emphasis on portability and low power draw [1]. Kurz et al. built a backpack worn detection system that covered the region from the head to the hip, combining ultrasonic sensing with vibration and audio feedback, and reported a detection rate of 94.4 percent during practical user testing [2]. Mocanu et al. went a step further by fusing ultrasonic sensor readings with smartphone camera based computer vision, allowing their system to identify both static and moving obstacles in outdoor urban scenes, and evaluated the result with twenty one visually impaired participants [3]. Together, these studies confirm that ultrasonic sensing remains a reliable and inexpensive foundation for obstacle detection, and this conclusion directly informed the decision to build Blind Mode around a three sensor ultrasonic array paired with a time to collision calculation rather than a more expensive depth camera or LIDAR unit.

### B. Offline and Real Time Captioning for the Deaf and Hard of Hearing

Most existing real time captioning systems for deaf and hard of hearing users have relied on cloud connected speech recognition services. Yamamoto et al. proposed a captioning system rendered on a transparent display, designed specifically to preserve eye contact and other non verbal cues during a live conversation, and built their pipeline on top of a cloud based automatic speech recognition engine [4]. More recent open source efforts, including the CapCap wearable captioning project, have started moving toward offline speech recognition models in order to remove both the latency and the privacy concerns that come with streaming a user's audio to a remote server. TriSense follows this offline first direction as a design principle rather than an afterthought, adopting the Vosk toolkit for speech recognition that runs entirely on the device, so that captioning keeps working even when no network connection is available at all.

### C. Sign Language Recognition Using MediaPipe and Lightweight Classifiers

Combining MediaPipe hand landmark extraction with a lightweight sequence classifier has become a common and practical approach to real time sign language recognition. Abdallah et al. paired MediaPipe landmark extraction with a combination of one dimensional convolutional layers and a GRU, specifically aiming for mobile and resource constrained devices, and reported strong results on a vocabulary of forty six dynamic signs [5]. Wang et al. compared LSTM and GRU classifiers operating on MediaPipe extracted joint coordinates using the LSA64 Argentine Sign Language dataset, reaching accuracies above 94 percent on a sixty four word vocabulary [6]. The WLASL dataset introduced by Li et al. provided the first large scale, word level American Sign Language video benchmark and is the dataset used to train the Mute Mode classifier described in this paper [7]. TriSense adopts the same MediaPipe plus convolutional plus GRU architecture family used by Abdallah et al., but scales the network down to roughly sixty eight thousand trainable parameters so that inference can run in real time on a Raspberry Pi 4 CPU rather than requiring a mobile GPU or a cloud backend.

### D. Positioning of This Work

Across all three domains described above, the literature shows mature and individually validated techniques, ultrasonic obstacle sensing, offline automatic speech recognition, and MediaPipe based sign language classification, but every one of these techniques appears in the literature as part of a separate, single purpose device or application. No system reviewed here attempts to combine all three into one switchable wearable. The contribution of TriSense is therefore mostly architectural rather than algorithmic. It takes three techniques that are each already well understood on their own and unifies them onto a single, low cost, fully offline Raspberry Pi 4 platform, coordinated by a resource aware mode switching layer that lets one physical device serve users with three different accessibility needs without duplicating hardware for each of them.

## III. System Requirements

Because TriSense is an embedded wearable device rather than a conventional client server application, the terms front end and back end are used here in a slightly adapted sense. The front end refers to the physical, user facing input and output channels of the device, while the back end refers to the on device processing logic that drives those channels.

### A. Front End: User Facing Interface

The front end of TriSense is built entirely from physical, low visual, or non visual output channels, since the device is meant to be operated without a conventional screen or touch interface. Directional haptic motors serve Blind Mode, delivering vibration pulses whose duration scales with how close an obstacle has come. A shared speaker and audio output channel serves both Blind Mode, for spoken obstacle alerts, and Mute Mode, for spoken output of recognized signs, and both are driven through a single shared text to speech engine so that the two modes never fight over ownership of the same audio hardware. A 128 by 64 pixel OLED display serves Deaf Mode as a compact, three line captioning viewport that keeps a short history buffer of recent transcript text. Finally, a single physical mode switch control lets the user cycle between Blind, Deaf, and Mute modes without needing to navigate any screen based menu.

### B. Back End: On Device Processing Logic

The back end runs entirely on the Raspberry Pi 4 and is organized as three independent, per mode pipelines that sit underneath one shared ModeManager layer. Sensor and signal acquisition is handled by dedicated drivers for the HC-SR04

ultrasonic sensors, the INMP441 I2S microphone stream, and camera frame capture for hand landmark extraction. Per mode processing logic includes closing speed and time to collision computation for Blind Mode, Vosk based offline speech recognition together with caption formatting for Deaf Mode, and MediaPipe landmark extraction feeding a convolutional and GRU classifier for Mute Mode. Sitting above these pipelines, the ModeManager and ResourceGuard layer starts and stops each mode's pipeline through thin adapter classes and enforces single owner access to the six shared hardware resources described earlier. A centralized text to speech worker service is shared between Blind Mode and Mute Mode so that only one component ever owns the speaker at a given time.

### C. Hardware and Software Bill of Materials

On the hardware side, the system is built around a Raspberry Pi 4 Model B acting as the edge AI compute core. Blind Mode uses three HC-SR04 ultrasonic distance sensors together with vibration motors and NPN transistor driver circuitry for haptic output. Deaf Mode uses an INMP441 I2S MEMS microphone for audio capture and an SSD1306 128 by 64 OLED display connected over I2C for caption rendering. A MAX98357A I2S audio amplifier and speaker serve as the shared audio output stage for both Blind Mode and Mute Mode. Mute Mode uses a camera module compatible with the Raspberry Pi CSI interface for video input.

On the software side, the system runs Raspberry Pi OS, a Debian based Linux distribution, with Python 3 as the primary implementation language. Sensor and motor control is handled through the RPi.GPIO library, paired with a software mock layer that lets the entire system be developed and exercised on a regular laptop before any physical component is connected. Deaf Mode depends on the Vosk offline speech recognition toolkit together with a compact English acoustic model, the `sounddevice` library for audio stream capture, and the `luma.oled` library for display rendering. Mute Mode depends on MediaPipe Hands for real time hand landmark extraction and PyTorch for training and running the CNN plus GRU sign language classifier. Text to speech synthesis across the whole system is handled by `pyttsx3`, which performs synthesis entirely offline. Together, these choices keep the full system runnable and testable on a laptop through mock hardware layers well before physical components are connected, and keep every software dependency free of any requirement for network access at run time.

## IV. System Architecture

TriSense runs on a single Raspberry Pi 4, which functions as the shared edge AI compute core for all three accessibility modes. Each mode, Blind, Deaf, and Mute, is implemented as an independent, self contained pipeline complete with its own drivers, its own processing logic, and its own dedicated test suite. Above these three pipelines sits a ModeManager component responsible for starting and stopping each pipeline safely, and a ResourceGuard component responsible for enforcing single owner access to six shared hardware resources. Those six resources are the speaker, the camera, the microphone, the display, the group of GPIO pins used by the ultrasonic sensors, and the group of GPIO pins used by the haptic motors.

A key architectural decision made early in the project was to wrap each mode's pipeline in a thin adapter class rather than modifying the internal code of any pipeline. Each adapter simply calls the existing `start()` and `stop()` entry points that the pipeline already exposes. This decision kept every mode's own test suite intact throughout integration and avoided introducing new bugs into modules that had already been built and verified in isolation. It also meant that the three team members responsible for Blind Mode, Deaf Mode, and Mute Mode could continue developing and testing their own pipelines independently while the integration layer was being built around them.

Mode switching itself is governed by a five step transition protocol implemented inside ModeManager. When the user requests a new mode, the system first stops whichever pipeline is currently running. It then releases every hardware resource that pipeline was holding. Next, it verifies that every resource required by the newly requested mode is currently free. If that verification step finds a required resource still held by another component, a `ResourceLockError` is raised immediately, before any resource is acquired, which guarantees that the system can never be left in a partially switched state. Only after verification succeeds does ModeManager acquire the needed resources and start the new mode's pipeline. This ordering, stop, release, verify, acquire, start, is what keeps hardware ownership consistent even when a user switches modes rapidly or when a fault occurs mid transition.

## V. Working Principle of Each Mode

### A. Blind Mode

Blind Mode polls its three HC-SR04 ultrasonic sensors every one hundred milliseconds. For each sensor, the system estimates closing speed as the change in measured distance between two consecutive readings divided by the polling interval, and then computes time to collision as the current distance divided by that closing speed, as shown in equation (1).

$$\mathrm{TTC} = \frac{d(t)}{v_{\text{closing}}}, \qquad v_{\text{closing}} = \frac{d(t-\Delta t) - d(t)}{\Delta t} \tag{1}$$

When the computed time to collision drops below a threshold of 1.5 seconds, the system pulses a directional haptic motor for a duration proportional to how urgent the obstacle is judged to be. That motor pulse is fired from a dedicated background thread specifically so that the main sensing loop is never blocked while a motor is vibrating, which matters because several sensors can trigger an alert within a very short window of each other. When time to collision drops further, below a tighter threshold of 1.0 second, the system also triggers a spoken audio alert through the shared text to speech engine, subject to a two second cooldown so that the same warning is not repeated dozens of times per second when an obstacle sits very close to the sensor array.

### B. Deaf Mode

Deaf Mode captures audio through a uniform `AudioStreamer` interface designed to accept input from either a live INMP441 microphone or a pre recorded WAV file. This abstraction allowed the entire captioning pipeline to be developed and exercised on a laptop long before any physical microphone hardware was available. Captured audio is handed to a Vosk `KaldiRecognizer` instance, which performs speech recognition entirely on device and emits both partial, in progress transcripts and final, committed transcripts as recognition proceeds. A caption formatter then wraps the recognized text to fit the twenty character wide, three line viewport offered by the OLED display, while quietly retaining the full transcript history in memory and showing a leading ellipsis whenever older text has scrolled out of the visible window. This design choice matters because it keeps the user aware, at all times, that the caption on screen is a partial view onto a longer transcript rather than the entire utterance. A caption renderer draws the formatted text to the physical OLED display, updating continuously as partial results arrive and committing each finalized line to history once Vosk confirms it.

### C. Mute Mode

Mute Mode captures a stream of camera frames and passes each one through MediaPipe Hands, which extracts twenty one three dimensional keypoints for every hand it detects in frame. Each frame's landmarks are then translated so that the wrist becomes the coordinate origin, and scaled according to the hand's own bounding box span, a normalization step that makes the resulting features invariant to where the hand happens to sit in the frame and how far it is from the camera. A rolling buffer holds thirty frames of these normalized landmarks and smooths them using an exponential moving average with a smoothing factor of 0.6. If a hand is briefly occluded during a gesture, the system holds the last valid landmark values for up to five frames before treating the occlusion as missing data, so short, incidental occlusions do not immediately corrupt an otherwise valid gesture sequence. Once a full thirty frame, one hundred twenty six feature sequence has accumulated, representing two hands, twenty one keypoints per hand, and three coordinates per keypoint, it is passed into a lightweight classifier made up of two one dimensional convolutional layers followed by a single layer GRU and a final linear classification head. The entire classifier is sized at roughly sixty eight thousand trainable parameters specifically so that it can run in real time on the Raspberry Pi 4 CPU rather than requiring a mobile GPU or a cloud backend. Once a sign is predicted, it is spoken aloud through the same shared text to speech service that Blind Mode also depends on.

## VI. Implementation, Testing, and Results

### A. Verification Approach

Every module built for this project was written to be testable independently of physical hardware. Ultrasonic, microphone, and camera inputs can each be mocked in software, which allowed the team to develop and validate the great majority of the system on a laptop before assembling any physical components. This approach was adopted deliberately, so that logic errors could be found and fixed while they were still cheap to fix, well before the harder and slower process of physical hardware bring up began.

### B. Blind Mode Verification

Blind Mode was verified through a deterministic synthetic simulation representing a subject walking toward the sensor array at a constant speed of one hundred centimeters per second, starting at a distance of two hundred centimeters and ending at ten centimeters. The simulation logged fifty seven sensor rows across this walk and confirmed that closing speed and time to collision computation, the haptic alert threshold, and the audio alert threshold all behaved exactly as expected at every logged distance. This same synthetic test was re run after a later hardware fix described below and continued to pass without modification.

### C. Deaf Mode Verification

Deaf Mode was verified through a modular test suite covering each stage of the pipeline separately, audio streaming, the Vosk recognition wrapper, caption formatting, OLED rendering, and the end to end main application loop that ties all four stages together. All tests in this suite pass at the time of writing.

### D. Mute Mode Verification

Mute Mode is fully integrated end to end into ModeManager, including its call path into the shared text to speech service, and this integration has been verified with a mocked classifier standing in for the trained model during automated testing. Separately from integration testing, the classifier itself has been trained on real data drawn from the WLASL dataset across a vocabulary of one hundred words, and currently reaches 33.33 percent validation accuracy on that vocabulary. Random guessing across one hundred classes would be expected to succeed roughly 1 percent of the time, so this result represents real learned structure rather than chance. At the same time, an accuracy of one correct prediction in every three attempts falls well short of what a dependable assistive communication device would need, and the team does not present it as production ready. The most likely explanation is data volume rather than model architecture. The WLASL downloader used in this project guarantees only a minimum of eight samples per class, which is a small number for training a sequence classifier from scratch, and improving accuracy would most likely require more training samples per class, a longer training schedule, or a larger backbone network. None of these improvements were permitted to draw time or engineering resources away from Blind Mode or Deaf Mode, in keeping with Mute Mode's status as a protected stretch goal rather than a hard deliverable.

TABLE I
DEVELOPMENT AND TEST STATUS BY MODE

| Aspect | Blind Mode | Deaf Mode | Mute Mode |
|---|---|---|---|
| Software status | Complete | Complete | Integrated |
| Automated tests | Synthetic suite, pass | Modular suite, pass | Integration test with mocked classifier, pass |
| Hardware tested | Not yet | Not yet | Not yet |
| Deliverable tier | Hard deliverable | Hard deliverable | Protected stretch goal |

### E. The GPIO Pin Conflict

One of the more instructive results produced during this project did not come from any runtime test at all, but from a documentation cross check. Before any hardware was connected, the team carried out a systematic audit of every GPIO pin assignment across all three modes, comparing each assignment against the shared hardware bill of materials. This audit surfaced a three way conflict. The pins originally assigned to the Blind Mode ultrasonic array, specifically ECHO_CENTER and ECHO_RIGHT, collided with the I2S bus lines shared by both the INMP441 microphone and the MAX98357A speaker amplifier. Had this conflict gone unnoticed, it would most likely have produced silent hardware failures once physical components were connected, showing up as intermittent sensor read errors or unexplained audio dropouts that would have been very difficult to trace back to a pin assignment problem through software behavior alone. The team reassigned the conflicting pins before any hardware bring up was attempted, moving ECHO_CENTER from GPIO19 to GPIO16 and ECHO_RIGHT from GPIO21 to GPIO17. The Blind Mode synthetic test suite was then re run and confirmed to still pass after the reassignment. This outcome illustrates the practical value of the strict, incremental, verify before proceeding discipline that the team followed throughout development, since a defect of this kind would not have been caught by unit tests running in isolation.

TABLE II
SHARED HARDWARE RESOURCES AND OWNING MODE

| Resource | Used By |
|---|---|
| Speaker | Blind Mode (audio alerts) and Mute Mode (sign to speech output) |
| Camera | Mute Mode |
| Microphone | Deaf Mode |
| Display | Deaf Mode |
| GPIO sensor lines | Blind Mode (ultrasonic array) |
| GPIO motor lines | Blind Mode (haptic motors) |

### F. Open Items at the Time of Writing

Two issues remain open and unresolved at the time of writing, neither of which affects the software test results reported above, since both concern behavior that only becomes visible once physical hardware is connected. First, ModeManager does not currently invoke the centralized text to speech worker's startup routine, which means spoken output may not function correctly end to end until this wiring gap is closed. Second, the pull down resistor and inter sensor delay fixes already documented in the Blind Mode hardware checklist have not yet been confirmed as applied inside the sensor driver code itself, and this confirmation is planned as one of the first steps of hardware bring up.

## VII. LIMITATIONS AND FUTURE WORK

The most significant limitation of the project at this stage is that none of the three modes have been validated on physical hardware. Every result reported in this paper rests on software simulation, mock hardware layers, or modular unit testing conducted on development laptops rather than on the assembled wearable device itself. This limitation is acknowledged deliberately rather than hidden, since the whole point of the verification approach used throughout the project was to catch and fix logic errors in software before the more expensive and less forgiving process of hardware bring up begins.

Several concrete next steps follow directly from this limitation. The team plans to procure and assemble the full hardware bill of materials described in Section III. The pending wiring gap that currently prevents ModeManager from starting the centralized text to speech worker needs to be closed and re tested. The pull down resistor and inter sensor delay fixes already written into the Blind Mode hardware checklist need to be confirmed as present in the actual sensor driver code. Finally, the hardware validation checklists already prepared for both Blind Mode and Deaf Mode need to be executed against the physical device once assembly is complete.

Beyond hardware bring up, the Mute Mode classifier remains an open area for improvement. Raising its accuracy above the current 33.33 percent would most plausibly come from collecting more training samples per class than the eight sample minimum currently guaranteed by the WLASL downloader, training for a longer schedule, or increasing the size of the backbone network. Any of these directions would need to be pursued without displacing time or resources committed to the two hard deliverables, consistent with how Mute Mode has been scoped from the beginning of the project.

## VIII. CONCLUSION

This paper presented TriSense, a single wearable edge AI device that offers blind, deaf, and mute users three independently switchable accessibility modes from one shared, fully offline Raspberry Pi 4 platform. Blind Mode and Deaf Mode, the two hard deliverables of the project, are software complete and have each been verified through dedicated synthetic or modular test suites that currently pass without failure. Mute Mode, scoped from the outset as a protected stretch goal, has been carried substantially further than originally planned. It is fully integrated into the mode coordination layer and has been trained on real WLASL data, while its 33.33 percent classification accuracy is reported honestly as not yet sufficient

for a dependable real world assistive device. The ModeManager and ResourceGuard coordination layer described in this paper successfully unifies all three independently developed pipelines onto a shared set of hardware resources without requiring any of their internal code to be rewritten, and a systematic documentation audit conducted before hardware bring up caught and resolved a three way GPIO pin conflict that could otherwise have caused difficult to diagnose hardware failures later in the project. The central risk still facing the project is that no physical hardware validation has yet taken place, and the next phase of work is oriented entirely around closing that gap. Even with that risk acknowledged plainly, the results presented here support the conclusion that a single, low cost, fully offline wearable device can plausibly serve blind, deaf, and mute users at once, so long as shared hardware access between its accessibility modes is coordinated carefully at the software level.

## ACKNOWLEDGMENT

The authors thank Dr. Bubai Manna of the School of Computer Science and Engineering, VIT Bhopal University, for supervising this capstone project under course code DSN4091.

## REFERENCES

[1] K. Sweekat, D. Mohammad, and M. Monif, "An Electronic Device to Assist the Blind Using Ultrasonic Sensors," in *Proc. Int. Conf. Biomedical Engineering (ICBME)*, Al-Andalus University, Tartus, Syria, 2022.

[2] J. Kurz, V. Tharmalingam, N. Pryakhina, A. Hunger, M. Ollenschläger, and B. Eskofier, "A wearable obstacle detection system for visually impaired people," in *Proc. 41st Annu. Int. Conf. IEEE Engineering in Medicine and Biology Society (EMBC)*, Berlin, Germany, 2019.

[3] B. Mocanu, R. Tapu, and T. Zaharia, "When Ultrasonic Sensors and Computer Vision Join Forces for Efficient Obstacle Detection and Recognition," *Sensors*, vol. 16, no. 11, art. 1807, 2016.

[4] K. Yamamoto, I. Suzuki, A. Shitara, and Y. Ochiai, "See-Through Captions: Real-Time Captioning on Transparent Display for Deaf and Hard-of-Hearing People," University of Tsukuba, 2021.

[5] M. S. Abdallah, G. H. Samaan, A. R. Wadie, F. Makhmudov, and Y.-I. Cho, "Light-Weight Deep Learning Techniques with Advanced Processing for Real-Time Hand Gesture Recognition," *Sensors*, vol. 23, no. 1, art. 2, 2023.

[6] Y. Wang, R. Li, and G. Li, "Sign language recognition using MediaPipe," unpublished, 2023.

[7] D. Li, C. Rodriguez, X. Yu, and H. Li, "Word-level Deep Sign Language Recognition from Video: A New Large-scale Dataset and Methods," in *Proc. IEEE/CVF Winter Conf. Applications of Computer Vision (WACV)*, 2020.

[8] Alpha Cephei, "Vosk Offline Speech Recognition Toolkit," Alpha Cephei Inc. [Online software toolkit].

[9] Google Research, "MediaPipe Hands: On-device Real-time Hand Tracking," Google AI. [Online software framework documentation].

[10] Raspberry Pi Foundation, *Raspberry Pi 4 Model B Product Documentation*, Raspberry Pi Foundation, Cambridge, U.K.

[11] B.-C. Chen, Y.-Y. Chen, Y.-H. Kuo, and W. Hsu, "Scalable Face Image Retrieval using Attribute-Enhanced Sparse Codewords," *IEEE Transactions on Multimedia*, vol. 3, no. 1, pp. 1–11, 2012.